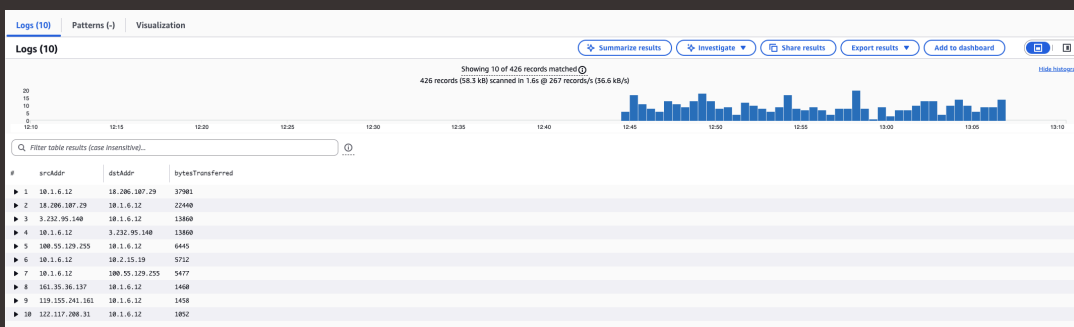




VPC Monitoring with Flow Logs



jjoseph1608@outlook.com





Introducing Today's Project!

What is Amazon VPC?

Amazon VPC is a service that lets you create isolated private networks within AWS, and it is useful because it gives you full control over your network environment including IP ranges, subnets, route tables, and security settings, keeping your cloud resources separate and secure

How I used Amazon VPC in this project

In today's project, I used Amazon VPC to create two isolated networks, connect them with a peering connection, and monitor the traffic flowing between them using VPC Flow Logs and CloudWatch

One thing I didn't expect in this project was...

One thing I didn't expect in this project was how much detail VPC Flow Logs captures about network traffic. Being able to see the exact source and destination IP addresses, the protocol, and whether traffic was accepted or rejected gave me a much clearer picture of what is actually happening inside a network

This project took me...

This project took me around 90 minutes to complete, including setting up the VPCs, configuring the peering connection, troubleshooting the routing issues, and analysing the flow logs in CloudWatch



In the first part of my project...

Step 1 - Set up VPCs

In this step, I will create two VPCs from scratch, because I need two isolated networks set up before I can configure peering and start monitoring the traffic flowing between them.

Step 2 - Launch EC2 instances

In this step, I will launch an EC2 instance in each VPC, because I need something running inside each network to generate and receive traffic when I test the peering connection and start capturing flow logs

Step 3 - Set up Logs

In this step, I will set up VPC Flow Logs and a CloudWatch Log Group to store the captured traffic data, because without these in place I have no way to record or analyse what traffic is flowing through my VPCs

Step 4 - Set IAM permissions for Logs

In this step, I will set up an IAM role that grants VPC Flow Logs permission to send data to CloudWatch, and then finish setting up the flow log on my subnet, because AWS services need explicit permissions to interact with each other and without this the flow log cannot write any records



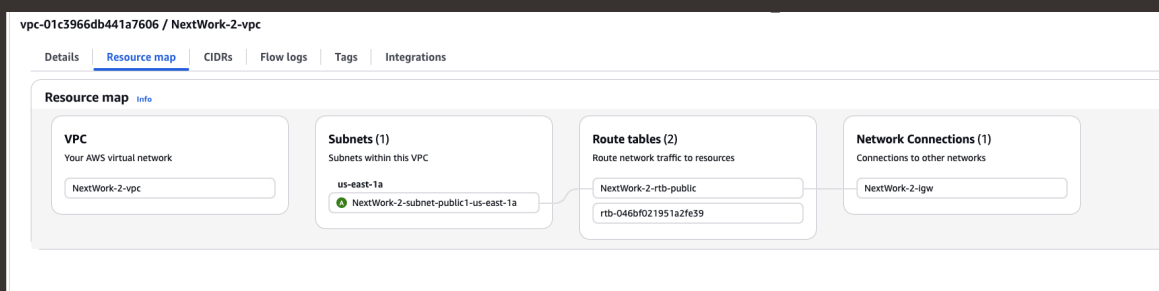
Multi-VPC Architecture

I started my project by launching two VPCs using the visual VPC resource map. I created two subnets in total, one public subnet inside each VPC

The CIDR blocks for VPCs 1 and 2 are 10.1.0.0/16 and 10.2.0.0/16. They have to be unique because if the IP ranges overlapped, the VPC peering connection would fail as AWS wouldn't be able to route traffic between them correctly

I also launched EC2 instances in each subnet

My EC2 instances' security groups allow all ICMP traffic from the CIDR block of the other VPC. This is because I need the instances to be able to receive ping requests from each other to test that the peering connection is working correctly





Logs

Logs are records of events and activity that happen within a system, in this case capturing details about the network traffic flowing in and out of my VPC such as the source, destination, and whether the traffic was accepted or rejected

Log groups are containers in Amazon CloudWatch that organise and store related log records together, making it easier to manage, search, and analyse them in one place

Successfully created flow log for the following resource:
vpc-07959a6a79255b9af

fl-03f181473f40a56a1 / NextWorkVPCFlowLog

Details

Flow Log ID
fl-03f181473f40a56a1

Name
NextWorkVPCFlowLog

State
Active

Creation Time
Thursday, June 4, 2026 at 13:43:35 GMT+1

Destination Type
cloud-watch-logs

Destination Name
NextWorkVPCFlowLogGroup

IAM Role
arn:aws:iam::304152263329:role/NextWorkVPCFlowLogRole

Cross Account IAM Role
-

Traffic Type
All

Max Aggregation Interval
1 minute

Log Format
Default

Tag field specifications
-

File Format
-

Hive Compatible Partitions
-

Partition Logs
-

Tags

Integrations

Tags

Search tags

Key	Value
Name	NextWorkVPCFlowLog

Manage tags



IAM Policy and Roles

I created an IAM policy because it defines exactly what actions VPC Flow Logs is allowed to perform in CloudWatch, such as creating log streams and writing log records, without it the service has no authorisation to send any data

I also created an IAM role because it is what gets attached to the VPC Flow Logs service to grant it the permissions defined in the IAM policy, the role is what AWS uses to identify which entity is allowed to perform those actions

A custom trust policy is a document that defines which AWS service or entity is allowed to assume an IAM role, in this case it tells AWS that the VPC Flow Logs service is trusted to take on the role and use its permissions

Custom trust policy
Create a custom trust policy to enable others to perform actions in this account.

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Sid": "Statement1",  
6       "Effect": "Allow",  
7       "Principal": {  
8         "Service": "vpc-flow-logs.amazonaws.com"  
9       },  
10      "Action": "sts:AssumeRole"  
11    }  
12  ]  
13 }
```



In the second part of my project...

Step 5 - Ping testing and troubleshooting

In this step, I will ping Instance 2 from Instance 1, because I need to generate traffic between the two VPCs so that the flow logs have real data to capture and I can confirm the peering connection is still working correctly

Step 6 - Set up a peering connection

In this step, I will set up a VPC peering connection between my two VPCs and update their route tables, because this is what allows traffic to flow between them over private IP addresses so my ping test can succeed and the flow logs can capture that traffic

Step 7 - Analyze flow logs

In this step, I will review the VPC Flow Logs in CloudWatch and use Log Insights to analyse the captured traffic data, because this is where I get to see everything that was recorded during my ping tests and understand what the logs are actually telling me about my network traffic

Connectivity troubleshooting

My first ping test between my EC2 instances had no replies, which means the traffic was not getting through, most likely because the security groups or route tables were not yet configured to allow ICMP traffic between the two VPCs

```

#
###
---
---
---
#
V-
---
---
---
m'
[ec2-user@ip-10-1-6-12 ~]$ ping 10.2.15.19
PING 10.2.15.19 (10.2.15.19) 56(84) bytes of data.

```

I could receive ping replies if I ran the ping test using the other instance's public IP address, which means the instances can reach each other over the public internet but the peering connection routing over private IPs is not yet working correctly



Connectivity troubleshooting

Looking at VPC 1's route table, I identified that the ping test with Instance 2's private address failed because there was no route pointing VPC 2's CIDR block through the peering connection, so the traffic had nowhere to go and was dropped

To solve this, I set up a peering connection between my VPCs

I also updated both VPCs' route tables so that each VPC knows to send traffic destined for the other VPC's CIDR block through the peering connection, without these routes the traffic would have no path to follow even with the peering connection in place

rtb-02b3cc32389e25ff7 / NextWork-1-rtb-public					
Details Routes Subnet associations Edge associations Route propagation Tags					
Routes (3)					
Filter routes					
Destination	Target	Status	Propagated	Route Origin	
0.0.0.0/0	igw-05b17317c3e7a0b38	Active	No	Create Route	
10.1.0.0/16	local	Active	No	Create Route Table	
10.2.0.0/16	pcx-00f1795d6a66c906	Active	No	Create Route	



Connectivity troubleshooting

I received ping replies from Instance 2's private IP address! This means the VPC peering connection is working correctly, the route tables are properly configured, and traffic is successfully flowing between the two VPCs over private IP addresses

```
--- 3.232.95.140 ping statistics ---
91 packets transmitted, 91 received, 0% packet loss, time 93617ms
rtt min/avg/max/mdev = 0.324/0.387/0.862/0.072 ms
[ec2-user@ip-10-1-6-12 ~]$ ping 3.232.95.140
PING 3.232.95.140 (3.232.95.140) 56(84) bytes of data.
54 bytes from 3.232.95.140: icmp_seq=1 ttl=126 time=0.332 ms
54 bytes from 3.232.95.140: icmp_seq=2 ttl=126 time=0.379 ms
54 bytes from 3.232.95.140: icmp_seq=3 ttl=126 time=0.339 ms
54 bytes from 3.232.95.140: icmp_seq=4 ttl=126 time=0.365 ms
54 bytes from 3.232.95.140: icmp_seq=5 ttl=126 time=0.362 ms
54 bytes from 3.232.95.140: icmp_seq=6 ttl=126 time=0.362 ms
54 bytes from 3.232.95.140: icmp_seq=7 ttl=126 time=0.353 ms
54 bytes from 3.232.95.140: icmp_seq=8 ttl=126 time=0.388 ms
54 bytes from 3.232.95.140: icmp_seq=9 ttl=126 time=0.334 ms
54 bytes from 3.232.95.140: icmp_seq=10 ttl=126 time=0.378 ms
54 bytes from 3.232.95.140: icmp_seq=11 ttl=126 time=0.390 ms
54 bytes from 3.232.95.140: icmp_seq=12 ttl=126 time=0.417 ms
54 bytes from 3.232.95.140: icmp_seq=13 ttl=126 time=0.371 ms
54 bytes from 3.232.95.140: icmp_seq=14 ttl=126 time=0.344 ms
54 bytes from 3.232.95.140: icmp_seq=15 ttl=126 time=0.361 ms
54 bytes from 3.232.95.140: icmp_seq=16 ttl=126 time=0.372 ms
54 bytes from 3.232.95.140: icmp_seq=17 ttl=126 time=0.375 ms
54 bytes from 3.232.95.140: icmp_seq=18 ttl=126 time=0.380 ms
54 bytes from 3.232.95.140: icmp_seq=19 ttl=126 time=0.372 ms
54 bytes from 3.232.95.140: icmp_seq=20 ttl=126 time=0.372 ms
```



Analyzing flow logs

Flow logs tell us about the source and destination IP addresses, the ports used, the protocol, how much data was transferred, and whether the traffic was accepted or rejected by the security group or network ACL

For example, the flow log I've captured tells us the source IP address of the traffic, the destination IP address, the protocol used, and whether the traffic was accepted or rejected, giving me a clear record of the ping requests that travelled between my two EC2 instances

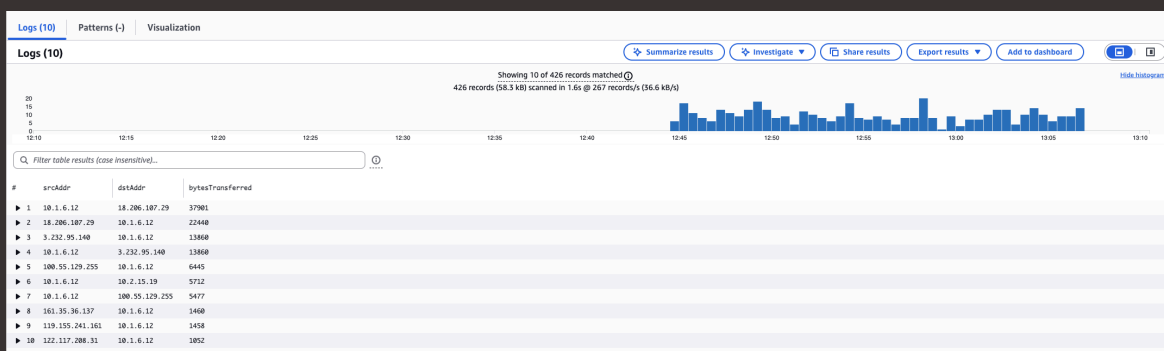
```
2 304152263329 eni-0342a587a7867a172 216.180.246.147 10.1.6.12 47530 53 17 1 64 1780578142 1780578163 REJECT OK
```



Logs Insights

Logs Insights is a feature in Amazon CloudWatch that lets you run queries against your log data to filter, aggregate and analyse it quickly, making it much easier to extract useful information from large volumes of log records without having to read through them manually

I ran the query `stats sum(bytes) as bytesTransferred by srcAddr, dstAddr | sort bytesTransferred desc | limit 10`. This query analyses the flow log data by adding up the total bytes transferred between each unique pair of source and destination IP addresses, sorts them from highest to lowest, and returns the top 10 results, giving me a clear picture of where the most traffic in my network was flowing





nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

